Dimensionality Reduction of Sparse Functional Data Using Transformers

Sri Pranav Kunda

March 2025

Contents

1. Dimensionality Reduction for Functional Data
2. NeuralInt for Dimensionality Reduction
3. Experiments

Dimensionality Reduction for Functional Data

Basis Expansions

Dimensionality reduction through basis expansions:

Given a basis of M continuous functions $\{\beta_i(t)\}$, we want to express our process as

$$X(t) \approx \sum_{i=1}^{M} b_i \beta_i(t),$$

after which the coefficient set $\{b_i\}$ becomes the data we are interested in.

Computing Coefficients

The coefficients b_i can be determined by the inner product

$$b_i = \langle \beta_i, X \rangle = \int_0^1 \beta_i(t) \cdot X(t)\, dt.$$

In practice, this integral is computed through numerical methods.

The numerical accuracy is heavily decreased for sparse functional data.

PACE (Functional Principal Component Analysis)

Functional principal component analysis involves estimating the principal component functions and their associated principal component scores.

PACE instead uses the Karhunen-Loève expansion and computes the principal component scores by using the best linear predictor model to avoid numerical integration. This works well for sparsely observed gaussian processes, but is less effective if X is more sophisticated.

NeuralInt for Dimensionality Reduction

SAND: Imputation of Sparse Functional Data

- The transformer architecture has shown promising results for imputing sparse functional data, but is too perturbed by noise in the observations achieve sufficient smoothness and accuracy.
- The SAND architecture involves a new layer containing Diff and Intg operators which are respectively applied to the transformer output.

SAND: Imputation of Sparse Functional Data

- The Diff operator is a multi-head self attention module.

$$\text{Diff}(\widetilde{T}) = \widetilde{D} = \sum_{h=1}^{H} W_O^{(h)} \left(W_V^{(h)} \widetilde{T}_c \right) \left[\left(W_K^{(h)} \widetilde{T}_c \right)^\top \left(W_Q^{(h)} \widetilde{T}_c \right) / \sqrt{h_d} \right].$$

- The Intg operator performs numerical integration. It does not contain learnable parameters.

Motivating NeuralInt

The NeuralInt architecture imputes sparse functional data in a manner similar to SAND but aims to do so by simultaneously recovering the integral function

$$\mathcal{X}(u) = \int_0^u X(t)\,dt.$$

We achieve this by keeping the transformer architecture and reversing the roles of the Diff and Intg operators.

The Intg Operator in NeuralInt

In NeuralInt, we define Intg as follows:

$$\mathsf{Intg}(t) = \mathrm{FNN}\left(\sum_{i=1}^{H} W_O(W_V^{(i)}E(t))\left[\frac{(W_K^{(i)})E(t)^\top(W_Q^{(i)}E(t))}{\sqrt{d/H}}\right]\right),$$

It is a multi-head self attention module similar to the Diff operator in SAND, where E is a learnable affine map $\mathbb{R} \to \mathbb{R}^d$.

The Diff Operator in NeuralInt

Unlike numerical integration, most deep learning pacakages provide ways of computing derivatives *a priori*. Therefore, we define Diff as the true derivative operator. Then the output from NeuralInt is

$$\text{NeuralInt}(t) = \mathsf{Diff}(\mathsf{Intg}(t)) = \frac{\partial \mathsf{Intg}}{\partial t}.$$

and the learned integral is conatined in Diff. It is important to note that Intg is trained alongside the transformer, but does not take as input the transformer's output.

Experiments

Setup and Architecture

We conduct the following experiments using the following configuration:

- **Transformer:** Hidden dimension of 512, feedforward dimension of 2048, 2 attention heads, and 6 encoder/decoder layers
- **NeuralInt:** Hidden dimension of 128, feedforward dimension of 512, and 2 attention heads.

Evaluating Imputation and Integration Capabilites (Setup)

We consider a signal whose integral function is known and verify that NeuralInt sufficiently learns this function:

$$X(t) = \sqrt{2}\sin(4\pi t) + \sqrt{3}\cos(6\pi t).$$

We sample this signal sparsely and add normally distributed noise to create the data.

Evaluating Imputation and Integration Capabilites (Results)

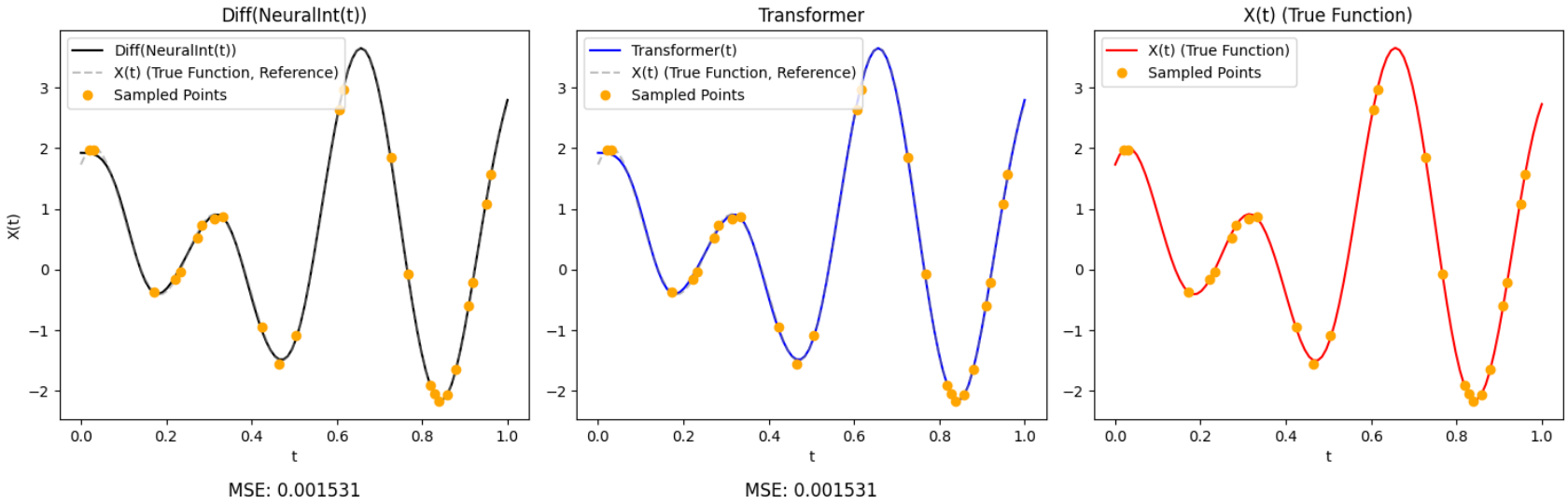


Figure: The imputed signal $X(t) = \sqrt{2}\sin(4\pi t) + \sqrt{3}\cos(6\pi t)$.

Evaluating Imputation and Integration Capabilites (Results)

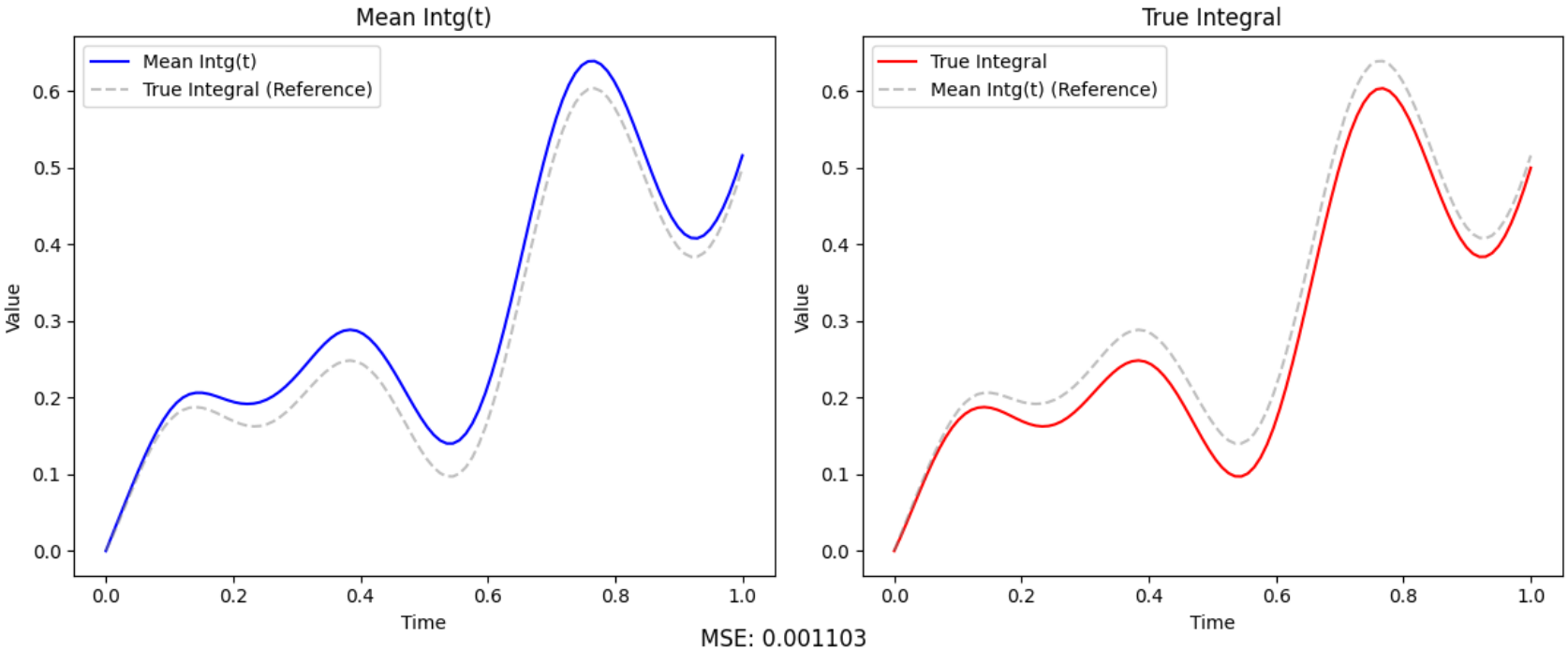


Figure: The learned integral of $X(t) = \sqrt{2}\sin(4\pi t) + \sqrt{3}\cos(6\pi t)$.

NeuralInt on COVID-19 Data (Setup)

- We want to evaluate the dimensionality reduction results of NeuralInt on real-world COVID-19 data.
- We sample the first 100 days of data and attempt to reduce express the data as a basis expansion of the Legendre polynomials $\{L_i\}$.

NeuralInt on COVID-19 Data (Setup)

- We choose the Legendre polynomials because they allow us to easily verify if NeuralInt successfully captures the orthogonality of basis functions.
- Since the COVID-19 data is already somewhat sparse, we induce only a slight artificial sparsity to evaluate the performance in a more relevant setting.

NeuralInt on COVID-19 Data (Results)

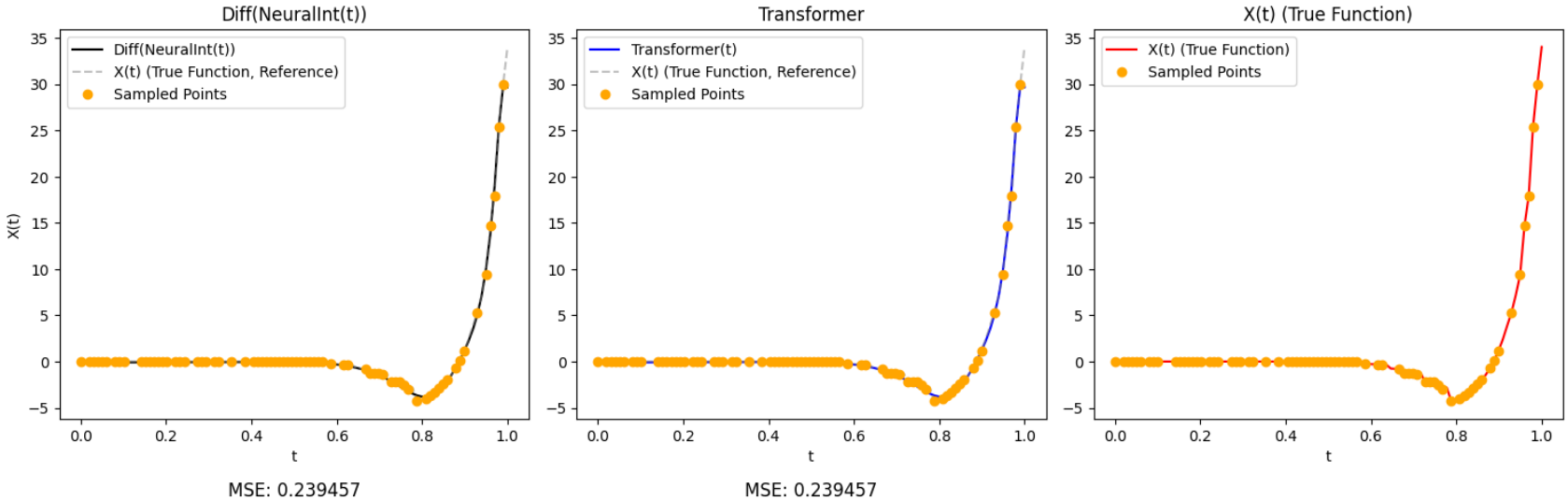


Figure: Example of coefficient computation for polynomial expansion.

NeuralInt on COVID-19 Data (Results)

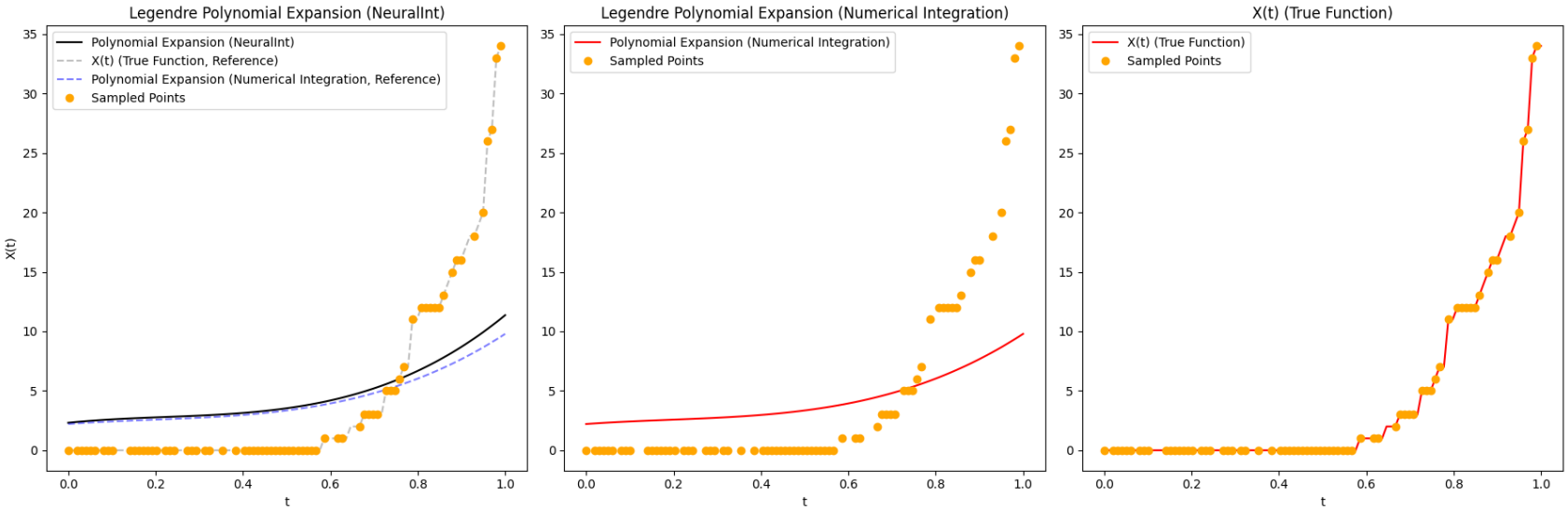


Figure: Legendre polynomial expansions of COVID data

Discussion and Further Work

NeuralInt has shown as an effective tool for computing integrals of sparsely observed functional data. In order to further explore the efficacy of the method, the following tasks are interesting:

- Compare the efficacy of FPCA (PACE) and NeuralInt on various gaussian and non-guassian processes
- Explore results using other basis functions (Fourier basis, B-splines, etc.)
- Explore how NeuralInt can be used to solve Neural Ordinary Differential Equations